Vehicle Parking Management System

MINI PROJECT USING C LANGUAGE

PARKINN: 05
Vehicle
Parking
Management System
P
1
3
6
4
3
4

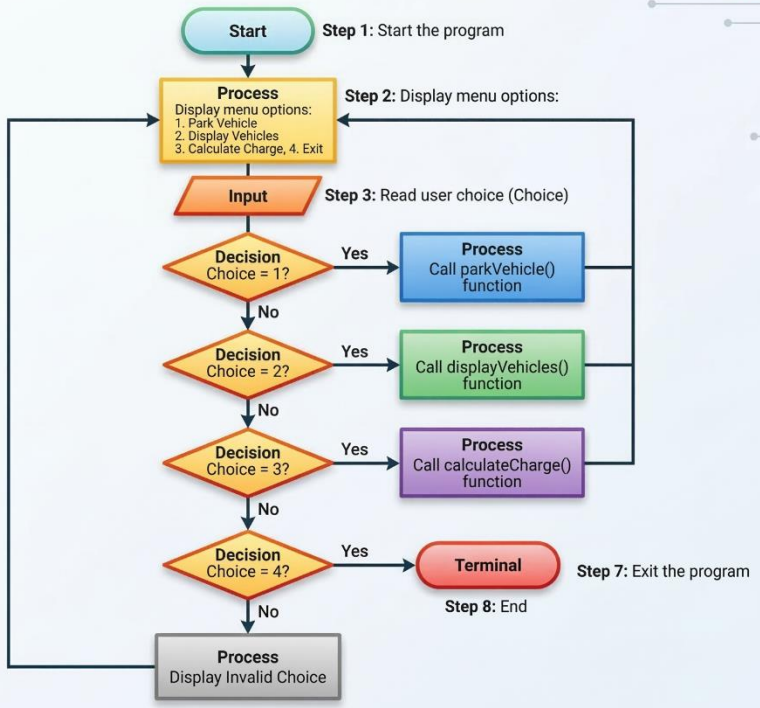
Introduction

Parking management is an important aspect in modern cities where large numbers of vehicles
require organized parking spaces. Manual parking management often leads to errors in vehicle racking and billing.
The Vehicle Parking Management System is designed to simplify the process of storing vehicle
information and calculating parking charges. Using C programming, the system stores data in files and retrieves it when required.
This project helps students understand practical implementation of programming concepts such as structures, file handling, and menu-driven programs.

Objectives

- To design a simple vehicle parking management system using C programming.
- To store vehicle information using file handling
- techniques.
- To manage different types of vehicles such as Bike,
- Car, and Truck.
- To calculate parking charges automatically based on
- vehicle type and hours.
- To understand the concept of modular programming.
- Software Requirements

Algorithm Flowchart: Vehicle Parking System



Software Requirements

Programming Language

- C Programming Language

Compiler

- GCC
- Turbo C
- Dev C++

Operating System

- Windows
- Linux

System Design

- The system is designed using a **menu-driven approach**.
- Users can select different options from the menu to perform various operations.

Main components of the system include:

- Vehicle Data Storage
- Vehicle Entry Module
- Vehicle Display Module
- Charge Calculation Module

Data Structure Used

The project uses a **structure** to store vehicle details.

```
struct Vehicle
{
   int number;
   char type[10];
   int hours;
};
```

Structure Fields

- **Vehicle Number** – Unique identification number of the vehicle
- **Vehicle Type** – Type of vehicle (Bike, Car, Truck)
- **Hours** – Duration of parking

Module Division

Student 1 – Data Structure and File Handling

- Create the **Vehicle structure** to store vehicle details.
- Create and manage the **parking data file**.
- Store vehicle records in the file using **file handling**.

Student 2 – Park Vehicle Module

- Take vehicle details from the user.
- Details include **vehicle number, type, and parking hours**.
- Save the entered vehicle data into the file.

Student 3 – Display Vehicles Module

- Read vehicle data from the file.
- Show all parked vehicles on the screen.
- Display vehicle number, type, and hours.

Student 4 – Parking Charge and Menu System

- Calculate parking charges based on **vehicle type and hours**.
- Connect all modules and run the program.

Algorithm

- Step 1: Start the program
- Step 2: Display menu options
- Step 3: Read user choice
- Step 4: If choice = 1
  - Call **parkVehicle()** function
- Step 5: If choice = 2
  - Call **displayVehicles()** function
- Step 6: If choice = 3
  - Call **calculateCharge()** function
- Step 7: If choice = 4
  - Exit the program
- Step 8: End

SMART PARKING AREA
SMART PARKING AREA
LEVEL 1-4
P101
P102
P103
P104
P105
P106
P107
P110
P111
P112
P113
PAYMENT CONFIRMED
AMOUNT: $12.50
PLATE: AB-1234
EXIT NOW
EXIT
AB-1234
EXIT
EXIT

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>


struct Vehicle

{

   int number;

   char type[10];

   int hours;

};


void parkVehicle()

{

   struct Vehicle v;
```

```
FILE *fp = fopen("parking.dat", "ab");

if (fp == NULL)
{
    printf("File error\n");
    return;
}

printf("Enter vehicle number: ");
scanf("%d", &v.number);

printf("Enter vehicle type (Bike/Car/Truck): ");
scanf("%s", v.type);
```

```
    printf("Enter parking hours: ");
    scanf("%d", &v.hours);

    fwrite(&v, sizeof(v), 1, fp);
    fclose(fp);

    printf("Vehicle parked successfully.\n");
}
```

```
#include <stdlib.h>
#include <string.h>
```

```
struct Vehicle
{
    int number;
    char type[10];
    int hours;
};

void parkVehicle()
{
    struct Vehicle v;
    FILE *fp = fopen("parking.dat", "ab");

    if (fp == NULL)
```

```
{
    printf("File error\n");
    return;
}

printf("Enter vehicle number: ");
scanf("%d", &v.number);

printf("Enter vehicle type (Bike/Car/Truck): ");
scanf("%s", v.type);

printf("Enter parking hours: ");
scanf("%d", &v.hours);
```

```
    fwrite(&v, sizeof(v), 1, fp);
    fclose(fp);

    printf("Vehicle parked successfully.\n");
}
```

Sample Output

Program Menu

--- VEHICLE PARKING SYSTEM ---

1. Park Vehicle
2. Display Parked Vehicles
3. Calculate Parking Charge
4. Exit

Example

Enter vehicle number: 1234
Enter vehicle type: Car
Enter parking hours: 5

Vehicle parked successfully.

Charge Calculation

Enter vehicle number: 1234
Total parking charge = Rs.100

Advantages

- Easy to use
- Simple implementation
- Fast execution
- File based data storage
- Easy to extend

Future Enhancements

- Add graphical user interface
- Add parking slot management
- Add entry and exit time tracking
- Add automatic billing system

Conclusion

- The **Vehicle Parking Management System** is a simple application developed using C programming.
- The system demonstrates the use of **structures, file handling, and menu-driven programming**.
- It efficiently stores vehicle records and calculates parking charges automatically.
- The project provides a practical understanding of **real-world problem solving using programming**.

Thank You

Submitted By:

S.Jayaram Siva Prasad (25A31A05GB)

D.Rohit (25A31A05FZ)

S.Sairam (25A31A05GA)

Nikhil (25A31A05FY)